

Assignment 4 Self Supervised Representation Learning for Time Series Classification

Core submission: thorough presentation of results and findings, reproducible code, saved configurations and machine-readable results

1 Main goal

The purpose of this assignment is to understand how self-supervised learning can produce useful representations of time series without class labels, and how those representations should be evaluated on downstream classification tasks.

You will study three methods: **T-Loss**, **TS2Vec** and **TF-C**. T-Loss and TS2Vec will be evaluated through frozen-representation classification on three benchmark datasets. TF-C will be examined through the paper's established industrial transfer task: self-supervised pretraining on FD-A bearing signals followed by bearing-condition classification on FD-B. You will also compare with raw-signal classifiers, examine performance when labels are scarce, test robustness to contiguous sensor dropout, and carry out one focused TS2Vec masking ablation.

Forecasting is outside the scope of this assignment.

2 Learning outcomes

After completing the assignment, you should be able to:

- explain the difference between temporal triplet learning, hierarchical contrastive learning and joint time-frequency contrastive learning;
- trace the main ideas in a paper to their implementation in an official repository;
- pretrain a time-series encoder without labels and evaluate it using frozen downstream probes;
- design a fair comparison under full- and limited-label settings;
- evaluate robustness to structured missing data; and
- interpret results critically, including cases where SSL does not outperform a strong raw-data baseline.

3 Methods and resources

T Loss required experiment

- Franceschi, Dieuleveut and Jaggi, *Unsupervised Scalable Representation Learning for Multivariate Time Series*, NeurIPS 2019: [paper](#)
- Official code: [GitHub](#) · [provided checkout](#)

TS2Vec required experiment

- Yue et al., *TS2Vec: Towards Universal Representation of Time Series*, AAAI 2022: [paper](#)
- Official code: [GitHub](#) · [provided checkout](#)

TF C required paper to code study and industrial experiment

- Zhang et al., *Self-Supervised Contrastive Pre-Training for Time Series via Time-Frequency Consistency*, NeurIPS 2022: [paper](#) · [supplement](#)
- Official code: [GitHub](#) · [provided unchanged checkout](#)

The supplied repositories are official checkouts. Keep the TF-C checkout unchanged for the required exploratory reproduction. Compatibility or integration code for T-Loss, TS2Vec or the optional FordA extension must be kept separate, documented and must not silently alter a method's learning objective.

4 Datasets

Use the official training and test partitions.

Dataset	Training / test series	Length × channels	Classes	Role
FordA	3,601 / 1,320	500 × 1	2	Main univariate industrial dataset
Articulatory Word Recognition	275 / 300	144 × 9	25	Small multivariate, many-class problem
Epilepsy	137 / 138	206 × 3	4	Small participant-disjoint motion dataset
FordB	3,636 / 810	500 × 1	2	Optional noisy-condition extension
FD-A	8,184 pretraining series	5,120 × 1	Labels not used	TF-C source bearing condition
FD-B	60 / 21 / 13,559	5,120 × 1	3	TF-C target bearing classification

The data are supplied in the [data folder](#). Official records are also available from the UCR/UEA Time Series Classification Archive:

- [FordA](#)
- [ArticulatoryWordRecognition](#)
- [Epilepsy](#)
- [FordB](#)
- [TF-C processed FD-A](#)
- [TF-C processed FD-B](#)

Epilepsy contains simulated seizure-like movement by healthy participants; it is not a clinical seizure-detection dataset. ArticulatoryWordRecognition does not include speaker identifiers, so its test performance must not be described as verified speaker-independent generalisation.

FD-A and FD-B contain vibration measurements from the same bearing-fault problem under different operating conditions. Their three target classes are undamaged bearing, inner-bearing damage and outer-bearing damage. In the TF-C experiment, FD-A labels must not be used during pretraining. The supplied FD-B partition contains 60 balanced training series, 21 balanced validation series and 13,559 test series.

5 Assignment tasks

Part A Understand the methods and connect paper to code

Read the method and experimental sections of the T-Loss, TS2Vec and TF-C papers. Explain the three learning ideas briefly in your own words and compare:

- what constitutes related views or positive examples;
- what information the objective encourages the encoder to preserve;
- how a fixed-length representation is obtained; and
- how the representation is used for classification.

Trace the following code elements in the supplied checkouts. For each one, identify (i) the closest paper location - an equation, algorithm, figure or section, as applicable; (ii) its inputs and outputs; (iii) its immediate caller or call site; and (iv) its role in training or representation extraction. If PyTorch invokes a `forward` method indirectly through `module(...)` or `nn.Sequential`, state that call site. An implementation helper will not always have its own paper equation; in that case, state this and connect it to the appropriate method step. The verified checkout commits are T-Loss [4aff592](#), TS2Vec [b0088e1](#) and TF-C [9667582](#).

T-Loss

- `losses/triplet_loss.py: TripletLoss.forward`

- `networks/causal_cnn.py`: `CausalConvolutionBlock.forward` and `CausalCNNEncoder.forward`
- `scikit_wrappers.py`: `TimeSeriesEncoderClassifier.fit_encoder` and `TimeSeriesEncoderClassifier.encode`

TS2Vec

- `ts2vec.py`: `TS2Vec.fit`, `TS2Vec._eval_with_pooling` and `TS2Vec.encode`
- `models/losses.py`: `hierarchical_contrastive_loss`, `instance_contrastive_loss` and `temporal_contrastive_loss`
- `models/encoder.py`: `TSEncoder.forward`, `generate_binomial_mask` and `generate_continuous_mask`

For T-Loss, show how `fit_encoder` selects the fixed-length or varying-length loss, how `TripletLoss.forward` samples reference, positive and negative intervals, and how `CausalCNNEncoder` produces one fixed-length vector. Confirm how to call `fit_encoder` without labels. For TS2Vec, trace the overlapping crops in `TS2Vec.fit`, state where timestamp masking is applied, map the two elementary losses and hierarchical pooling to Equations 1-3 and Algorithm 1, and identify the `TS2Vec.encode` option that returns one vector per complete series.

TF-C

- `code/TFC/dataloader.py`: `Load_Dataset.__init__`, `Load_Dataset.__getitem__`, `data_generator` and the `fft.fft(...).abs()` operation
- `code/TFC/augmentations.py`: `DataTransform_TD`, `DataTransform_FD`, `remove_frequency` and `add_frequency`
- `code/TFC/model.py`: `TFC.__init__`, `TFC.forward` and `target_classifier.forward`
- `code/TFC/loss.py`: `NTXentLoss_poly.forward`
- `code/TFC/trainer.py`: `Trainer`, `model_pretrain`, `model_finetune` and `model_test`
- `code/TFC/main.py`: the subset setting, call to `data_generator` and pretrained-checkpoint loading

Follow one TF-C batch from the time-domain signal to its frequency-domain version, augmented views, two encoders, projectors, pretraining losses and downstream classifier. Answer the following questions from the paper and the active, uncommented execution path:

- In `Load_Dataset.__init__`, along which tensor dimension is the FFT taken? Is the complex spectrum passed onward directly, or is it converted using magnitude, phase, real part or imaginary part? Is `fft` or `rfft` used?
- Which active time-domain augmentation is returned by `DataTransform_TD`? Which two frequency operations are called by `DataTransform_FD`, and how are their outputs combined? Do not describe commented alternatives as executed code.
- Which attributes in `TFC.__init__` define the separate time encoder, frequency encoder and two projectors? Relate the four outputs of `TFC.forward` to the paper's h and z notation.
- In `model_pretrain`, identify the calls that calculate the within-time loss, within-frequency loss, direct time-frequency term and the additional consistency expression. Which variables are actually included in the final `loss` passed to `backward()`? Compare this line explicitly with Equations 1-4 of the paper.
- In `model_finetune` and `model_test`, which tensors are concatenated for the downstream classifier? Use the optimizer calls and parameter updates, not only comments, to determine whether the encoder is frozen or fine-tuned.
- Trace `main.py` -> `data_generator` -> `Trainer` for FD-A to FD-B. Report the actual number of source and target-training samples used, whether `FD_B/val.pt` is loaded, how often the test loader is evaluated, and whether test results influence the reported best result. Record discrepancies from the paper's stated protocol; do not silently repair or reinterpret them.
- Inspect `remove_frequency` and `add_frequency` for device-specific tensor construction. State the computing-device assumption made by the supplied code and explain any failure observed in a CPU-only environment.

Part B Establish raw signal baselines

For FordA, ArticulatoryWordRecognition and Epilepsy, train:

1. logistic regression on the vectorised raw signal; and
2. an RBF-kernel SVM on the vectorised raw signal.

Evaluate two label regimes:

- **100% labels**: all official training labels;
- **10% labels**: a stratified subset of the official training labels.

Use three fixed random seeds. The 10% labelled subset generated by a seed must be reused by every method. Select preprocessing and classifier hyperparameters using training data only.

Report test accuracy and macro-F1 as mean $\pm$ standard deviation over the paired seeds. Retain both metrics even when they lead to different conclusions.

Part C Pretrain and evaluate T Loss and TS2Vec

For each required dataset and seed:

1. pretrain T-Loss and TS2Vec on the **unlabelled official training series**;
2. extract one fixed-length representation per series;
3. freeze the encoder;
4. fit the same downstream probe family on the 10% and 100% labelled training representations; and
5. evaluate once on the official test set.

The encoder may use every official training input during SSL pretraining in both label regimes, but it must not use their class labels. State clearly which probe is treated as the principal representation test. If both logistic regression and RBF-SVM are used as probes, report them separately; do not call the nonlinear SVM a linear probe.

Compare learned-representation results with both raw-signal baselines. Do not assume that SSL must win on every dataset.

Part D Robustness to contiguous sensor dropout

Create a corrupted version of each official test set by removing one or more contiguous intervals from each series. Choose a fixed dropout proportion and replacement convention before examining the results, justify them briefly, and apply exactly the same corrupted inputs to all methods.

Using the 100%-label models, compare clean and corrupted test performance. Report both the corrupted score and the change relative to the clean score. Discuss whether any apparent robustness comes from the representation, the classifier or the preprocessing convention.

Part E Focused TS2Vec continuous masking ablation

The default TS2Vec encoder uses binomial timestamp masking during training, while `models/encoder.py` also supplies `generate_continuous_mask`.

On **FordA**, compare:

1. the standard TS2Vec masking configuration; and
2. TS2Vec trained with continuous masking.

Keep the encoder, training budget, downstream probe, label regime and three seeds matched. Evaluate both versions on the clean and contiguous-dropout test sets. Determine whether continuous masking changes:

- clean classification performance;
- corrupted classification performance; and
- sensitivity to corruption relative to the model's own clean score.

This is a controlled ablation, not automatically a methodological innovation. Interpret a small, mixed or negative result honestly.

Part F Explore TF C for industrial bearing fault classification

Use the supplied, unchanged TF-C repository and its processed FD-A and FD-B data to execute the existing **FD-A to FD-B** workflow. This is an exploratory reproduction of the paper's one-to-one transfer scenario, not a request to reimplement TF-C.

The task is to determine whether a representation pretrained without FD-A labels can be used to classify the FD-B bearing state as undamaged, inner damage or outer damage. Complete the following:

1. Run the repository's documented TF-C pretraining stage with FD-A as the source and FD-B as the target.
2. Run its downstream fine-tuning and testing stage using the same seed and the checkpoint produced in Step 1.
3. Record the commands, environment, hardware, seed, data actually consumed, training time and reported classification metrics.

4. Present the FD-B accuracy and macro-F1 produced by the run and compare them cautiously with Table 5 of the TF-C supplement. Exact numerical reproduction is not required.
5. Explain what was transferred from FD-A, where FD-B labels entered the pipeline, and whether the final encoder was frozen or updated.
6. Use your Part A inspection to state any material difference between the paper's protocol and the supplied execution path, including subset use, loss composition, validation use or test-set access. Do not alter the TF-C learning objective or claim that an exploratory run is an exact reproduction.

One complete, documented seed is required for this exploratory task; further seeds are optional. Use a computing environment capable of running the supplied implementation. If execution is blocked, show the instructor the exact failure before substituting a different implementation or protocol.

Part G Synthesis

Provide a concise comparison across datasets, label regimes, methods and corruption conditions. Your discussion should address:

- when SSL provides a useful representation and when raw nonlinear classification remains competitive;
- whether the effect of SSL becomes clearer with only 10% labels;
- whether conclusions are stable across the paired seeds;
- how dataset size, dimensionality and number of classes may explain differences;
- whether continuous masking improves accuracy, robustness, both or neither;
- what the TF-C bearing experiment shows about transferring representations between operating conditions; and
- what these experiments suggest for future multichannel industrial sensor data.

6 Experimental rules

- Preserve every official test set as held-out data.
- Do not use test inputs for SSL pretraining in the required experiments.
- Fit normalisation, imputation and any feature transformation on training data only.
- Make the 10% label subsets stratified where possible and save their indices.
- Use the same seeds, label subsets, corruptions and probe settings in paired comparisons.
- Tune on a training-only validation split or cross-validation; never tune on test performance.
- In Parts C-E, separate representation extraction from probe training and keep the encoder frozen. In Part F, follow the supplied TF-C downstream procedure and state whether it fine-tunes the encoder.
- Record configurations, software versions, runtime and hardware.
- Report implementation departures from the official repositories.

7 Optional extensions

TF C on FordA

Optionally adapt TF-C to FordA without modifying the official checkout: place any loader, configuration or wrapper in your own integration code. Pretrain only on the unlabelled FordA training inputs, freeze the resulting encoder and use the same logistic-regression probe, 10% and 100% labelled subsets, seeds and official test split used for T-Loss and TS2Vec. Report a matched FordA table for T-Loss, TS2Vec and TF-C. Clearly separate this same-dataset frozen-probe experiment from TF-C's required FD-A to FD-B full-fine-tuning experiment.

FordB noisy condition extension

After completing the core assignment, repeat a selected subset of the experiments on FordB. Its test series were collected under noisier conditions than its training series, making it useful as an additional distribution-shift test.

Use FordB's own official train/test split and explain exactly which experiments were repeated. Do not concatenate FordA and FordB or assume that they form paired instances.

8 Submission

Submit:

1. **A thorough presentation of the results and findings** containing the method comparison, paper-to-code map, experimental protocol, principal result tables and plots, interpretation, limitations and conclusions. The slides should form a coherent account of what was done and what was learned.
2. **Reproducible code** that runs the raw baselines, SSL pretraining or loading, representation extraction, probe fitting, corruption experiment and masking ablation.
3. **A README** with environment setup and commands required to reproduce each result table or figure.
4. **Machine-readable outputs** containing per-seed results, configurations and saved label-subset indices.

The work will be assessed on conceptual understanding, validity and fairness of the experiments, quality of analysis, and reproducibility. Successful execution alone is not sufficient, and an SSL method failing to beat a baseline is not by itself a poor result.

9 Expected result table structure

At minimum, provide:

- clean-test results for every benchmark method and label regime;
- a clean-versus-dropout table for the 100%-label models;
- a FordA table for the standard-versus-continuous TS2Vec ablation;
- an FD-A to FD-B TF-C execution record and classification result; and
- a brief record of training time and representation dimensionality.

Put individual seed results in supplementary slides or machine-readable outputs; present mean $\pm$ standard deviation in the main presentation.